

Solidity智能合约开发

从入门到精通

第3.1课 ： 数组



今天的学习内容

1



数组类型：定长 vs 动态

2



数组基本操作

3



数组访问和修改

4



删除数组元素

5



多维数组

6



Gas优化技巧

7



安全使用数组

什么是数组？



数组的定义

存储相同类型元素的集合



定长数组



- ✓ 长度固定，不可改变
- ✓ 可以直接初始化
- ✓ 所有元素默认值为 0



动态数组



- ✓ 长度可变
- ✓ 可以使用 push 添加元素
- ✓ 可以使用 pop 删除元素

定长数组 (Fixed-size Array)

</> 声明方式

```
uint[5] public fixedArray;  
uint[3] public numbers = [1, 2, 3];
```

-  长度在声明时确定，且不可改变

≡ 特点

-  长度在声明时确定，且不可改变
-  可以直接进行初始化
-  所有元素若未显式赋值，默认值为其类型的零值（例如，uint类型默认为0）
-  不支持使用push()或pop()方法来添加或移除元素

💡 使用场景

-  当元素的数量是固定且已知时，例如表示一周的7天或一年的12个月份
-  不需要在合约执行过程中动态增删元素的场景

动态数组 (Dynamic Array)

</> 声明方式

```
uint[] public dynamicArray;
```

```
uint[] public initialized = [1, 2, 3];
```

📡 特点

- ✂ 长度可变，可以在运行时调整
- ➕ 支持使用`push()`方法添加元素
- ➖ 支持使用`pop()`方法移除最后一个元素
- 📏 拥有`length`属性，可获取当前长度
- ⚡ 提供更大的灵活性，但Gas成本相对较高

💡 使用场景

- ✅ 元素数量不确定，需要根据业务逻辑动态增删
- ✅ 用户列表、订单记录等动态数据集合

📄 动态数组操作

10 20

`pop()`

定长数组 vs 动态数组

特性	定长数组	动态数组
声明	uint[5]	uint[]
初始化	[1, 2, 3]	[1, 2, 3]
长度	固定	可变
push	✗	✓
pop	✗	✓
length	常量	可变属性
Gas成本	相对低	相对高
使用场景	固定数量	动态数量

数组基本操作

+ 添加元素

仅动态数组支持

```
arr.push(value);
```

🗑 删除最后元素

仅动态数组支持

```
arr.pop();
```

📏 获取长度

所有数组类型

```
arr.length
```

🔍 访问元素

所有数组类型

```
arr[0], arr[1], arr[2]
```

✏ 修改元素

所有数组类型

```
arr[0] = 100;
```

🚫 清空数组

所有数组类型

```
delete arr;
```

Memory 中创建数组

Memory 数组特点



必须在创建时指定长度
使用 `new` 关键字创建



不支持 `push` 或 `pop` 操作
长度一旦确定不可改变



所有元素初始化为默认值
例如 `uint` 类型为 0



临时存在，仅在函数执行期间有效
函数执行完毕后即被销毁

</> 代码示例

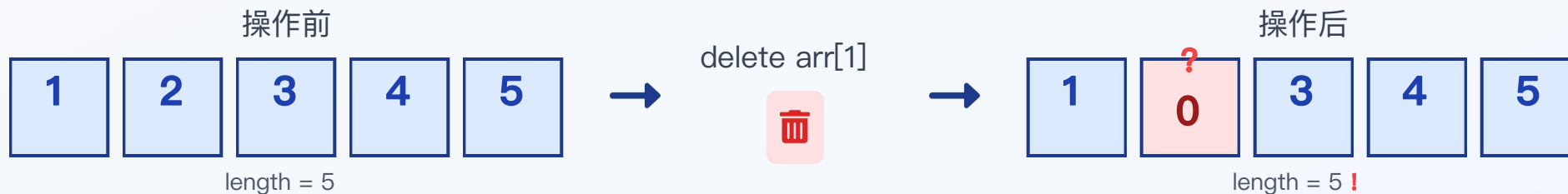
```
function createArray() public pure returns (uint[] memory) {  
  
    uint[] memory arr = new uint[](5);  
    arr[0] = 1;  
    arr[1] = 2;  
    arr[2] = 3;  
    arr[3] = 4;  
    arr[4] = 5;  
    return arr;  
}
```

Memory 数组可视化



delete 操作的陷阱

⚠ delete 操作的陷阱



✓ delete的效果

- 将元素设为默认值（例如uint为0）
- **不改变**数组的实际长度
- 不移动后续元素以填补空缺
- 留下"空洞"导致逻辑错误

❗ 常见问题

- 遍历时可能处理不必要的默认值
- 数组长度不能反映实际元素数量
- 可能导致Gas浪费
- 与期望的"移除元素"行为不符

正确删除数组元素

1 保持顺序（移动元素）

原始数组：

1	2	3	4	5
---	---	---	---	---

删除索引1后：

1	0	3	4	5
---	---	---	---	---

移动元素后：

1	3	4	5	5
---	---	---	---	---

```
for(uint i = index; i < arr.length - 1; i++) {  
    arr[i] = arr[i + 1];  
}  
arr.pop();
```

✓ 优点

保持数组元素的相对顺序

❗ 缺点

Gas成本较高，需移动多个元素

2 快速删除（不保序）

原始数组：

1	2	3	4	5
---	---	---	---	---

替换元素后：

1	5	3	4	5
---	---	---	---	---

移除最后一个元素：

	5	3	4	-
--	---	---	---	---

```
arr[index] = arr[arr.length - 1];  
arr.pop();
```

✓ 优点

Gas成本便宜，只需两步操作

❗ 缺点

不保持数组元素的原始顺序

数组遍历

🔄 基本遍历方法

使用for循环遍历数组元素：

```
for(uint i = 0; i < arr.length; i++) {  
    // 操作 arr[i]  
}
```

1

2

3

4

5

索引 i 从 0 到 length-1

☰ 常见操作

+ 计算总和

```
function sum() public view returns (uint) {  
    uint total = 0;  
    for(uint i = 0; i < numbers.length; i++) {  
        total += numbers[i];  
    }  
    return total;  
}
```

🔍 查找元素

```
function contains(uint value) public view  
returns (bool) {  
    for(uint i = 0; i < numbers.length; i++) {  
        if(numbers[i] == value) {  
            return true;  
        }  
    }  
    return false;  
}
```

🔿 过滤元素

```
function filter(uint threshold) public view  
returns (uint[] memory) {  
    uint[] memory result = new  
    uint[](numbers.length);  
    uint count = 0;  
    for (uint i = 0; i < numbers.length; i++)  
        if (numbers[i] > threshold) {  
            result[count] = numbers[i];  
            count++;  
        }  
    }  
    return result;  
}
```

📊 统计数量

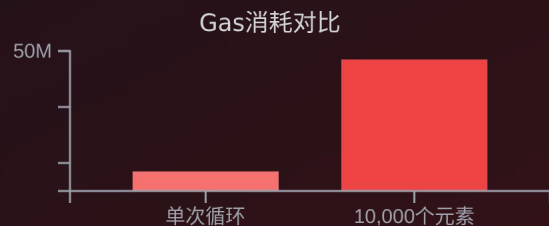
```
function count(uint value) public view  
returns (uint) {  
    uint count = 0;  
    for(uint i = 0; i < numbers.length; i++) {  
        if(numbers[i] == value) {  
            count++;  
        }  
    }  
    return count;  
}
```



数组遍历的风险

🔍 问题分析

- 💰 假设数组有 10,000 个元素
- ⚡ 每次循环消耗 5,000 gas
- 📊 总需求: 50,000,000 gas
- ⚠️ 超过区块 gas 限制!



💀 后果

- ✖ 函数永远无法执行
- ✖ 合约陷入"僵死"状态
- ✖ 用户资金可能被锁定

🕒 真实案例

多个项目因大数组遍历导致合约失效

💡 避免在单次交易中处理整个大型数组

多维数组 (Multi-dimensional Arrays)



声明顺序特殊

Solidity中数组声明顺序与传统编程语言**相反**!

`uint[3][4]` 表示**4个长度为3的数组**，不是3×4矩阵

</> 动态二维数组

```
uint[][] public matrix;
```



声明时**无需指定大小**



类似动态数组的二维形式



定长二维数组

```
uint[3][4] public fixed;
```



编译时确定大小



4个长度为3的uint数组



操作示例

```
matrix.push([1, 2, 3]);  
matrix.push([4, 5, 6]);  
uint value = matrix[0][1];
```

内存结构可视化

[	1	2	3	]
[	4	5	6	]



`matrix[0][1]` 访问结果为 2

优化技巧1: 缓存 length

✖ 不好的做法

```
function badSum() public view returns (uint) {  
    uint total = 0;  
    for(uint i = 0; i < data.length; i++) {  
        // 每次循环都读取storage  
  
        total += data[i];  
    }  
    return total;  
}
```

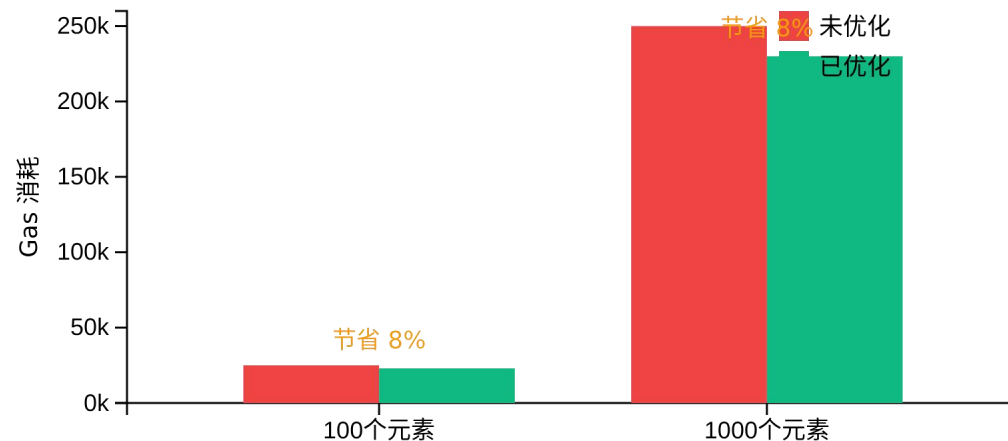
⚡ 每次循环读取 data.length (storage) 约 200 gas × 循环次数

✓ 优化后

```
function goodSum() public view returns (uint) {  
    uint total = 0;  
    uint len = data.length;  
    // 只读取一次length  
  
    for(uint i = 0; i < len; i++) {  
        total += data[i];  
    }  
    return total;  
}
```

⚡ 只读取一次 length, 节省约 200 gas × (循环次数-1)

Gas 优化效果



i 即使简单的缓存 length 操作, 也能在数组规模增大时带来可观的 Gas 节省

💡 对于大型数组 (>100个元素), 优化效果最为明显

优化技巧2：限制数组大小

为什么限制数组大小？

- ✓ 防止数组无限增长，避免Gas耗尽风险
- ✓ 使Gas成本更可预测，提高交易可靠性
- ✓ 防止因大数组遍历导致的区块Gas限制超限

// 设置最大长度常量

```
uint public constant MAX_SIZE = 100;
```

```
function safePush(uint value) public {  
    require(data.length < MAX_SIZE, "Array is full");  
    data.push(value);  
}
```

建议的最大长度



简单操作
≤ 1000 个元素

1000



复杂操作
≤ 100 个元素

100



安全起见
≤ 100 个元素

100



风险提示

不限制数组大小可能导致Gas耗尽，使交易回退，合约无法继续执行

优化技巧3：分批处理

⚠️ 大数组遍历风险

在单次交易中遍历大型数组可能导致Gas耗尽，超出以太坊区块的Gas限制，导致交易失败。

✅ 分批处理解决方案

通过实现分批处理函数，允许用户分多次交易处理数组的不同部分，将Gas消耗控制在区块Gas限制内。

★ 优势

- 可控的Gas消耗
- 避免Gas耗尽问题
- 可分多次交易处理
- 提高交易成功率

```
function sumRange(uint start, uint end) public view returns (uint) {  
    require(start < end);  
    require(end <= data.length);  
    uint total = 0;  
    for(uint i = start; i < end; i++) {  
        total += data[i];  
    }  
    return total;  
}
```

solidity

</> 使用方法

sumRange(0, 20)

sumRange(20, 40)

sumRange(40, 60)

...

1

2

3

4

5

6

7

8

9

...

■ 批次1 (0-20) ■ 批次2 (20-40) ■ 批次3 (40-60)

优化技巧4：使用 calldata

什么是 calldata?

calldata 是 Ethereum 中的一种只读数据区域，类似于 memory，但有重要区别：

- 只读 (immutable)
- 不需要将数据复制到内存中
- 对于大型数组，Gas 成本显著更低
- 适用于外部函数参数

代码示例

不好的做法：

```
function process(uint[] memory arr) public { ... }
```

优化后的做法：

```
function process(uint[] calldata arr) external { ... }
```

主要优势



节省 Gas 成本

不需要将数据从 calldata 复制到 memory，尤其对于大型数组，节省效果显著



只读保证

calldata 是只读的，编译器可以提供额外的优化保证



提高性能

减少内存操作，提高函数执行效率

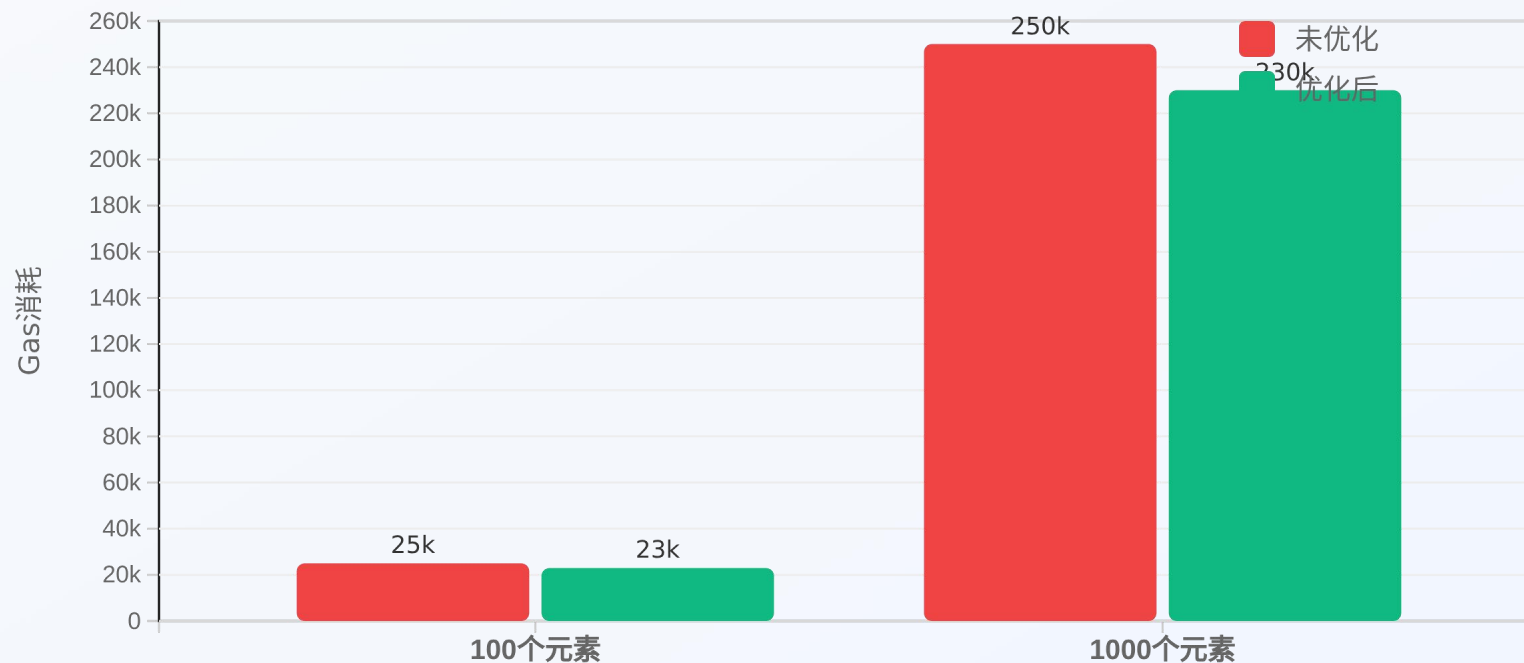


适用场景

- 外部调用的函数 (external)
- 不需要修改数组内容的场景
- 追求 Gas 优化的场合
- 大型数组参数处理

Gas优化效果对比

💡 缓存 **length** 属性可以显著减少Gas消耗，尤其在处理大型数组时



★ 关键发现

- 优化效果随数组规模增大而更加显著
- 即使是简单的缓存操作，也能带来可观的Gas节省
- 对于大型数组，优化效果可达总Gas消耗的8%

何时用数组？何时用 mapping？

选择数据结构

数组

有序集合，支持随机访问，适合小型数据集

使用数组的场景

- ✓ 元素数量少 (< 100)
- ✓ 需要遍历所有元素
- ✓ 需要保持顺序
- ✓ 需要获取所有数据

映射

键值对存储，快速查找，适合大型数据集

使用映射的场景

- ✓ 元素数量多或不确定
- ✓ 只需按键查找
- ✓ 不需要遍历
- ✓ 频繁查找操作

⚖️ **核心权衡：** 数组和映射之间的选择主要在于**可遍历性**与**查找效率**的权衡。

⚠ 数组使用的常见陷阱



陷阱1: delete不改变length

delete操作将元素设为默认值，但不改变数组长度，会在数组中留下"空洞"

```
delete arr[1]; // arr[1]=0, 但length不变
```



陷阱2: 越界访问

访问超出数组有效索引范围的元素会导致交易回退

```
require(index < arr.length); // 总是检查索引
```



陷阱3: 大数组遍历

遍历大型数组可能导致Gas耗尽，使交易失败

```
// 危险
for(uint i = 0; i < bigArray.length; i++) {
    require(bigArray[i] > 10);
}
```



陷阱4: 循环中写storage

在循环中频繁修改storage变量是昂贵的操作

```
// 危险
for(uint i = 0; i < values.length; i++) {
    data.push(values[i]); // 每次循环都写storage
}
```



陷阱5: memory数组push

memory中创建的数组是定长的，一旦创建长度不可变，不能使用push或pop

```
// 错误
uint[] memory arr = new uint[](5);
arr.push(1); // 编译错误: 不存在push方法
```

数组使用最佳实践



1 限制数组大小

设置最大长度常量 (MAX_SIZE)，防止数组无限增长，避免Gas耗尽风险。简单操作 ≤ 1000 ，复杂操作 ≤ 100 ，安全起见 ≤ 100 。



2 缓存length

在循环中将arr.length缓存到局部变量（如uint len = arr.length;），避免重复读取storage中的length值，节省约 $200\text{gas} \times (\text{循环次数} - 1)$ 。



3 检查越界

访问或修改数组元素前，始终使用require(index < arr.length, "Index out of bounds");验证索引有效性，防止交易回退。



4 分批处理

避免单次交易处理整个大型数组。设计接受start和end索引的函数，允许用户分批调用，将Gas消耗分散到多笔交易中。



5 考虑用mapping替代

如果业务逻辑主要涉及通过键进行查找，且不需要遍历所有元素，优先考虑使用mapping而不是数组。mapping在查找效率上具有显著优势。



6 使用calldata

对于外部函数接收的数组参数，如果函数内部不需要修改该数组，应将其声明为calldata（如function process(uint[] calldata arr) external）。

实战：用户管理系统

需求 • 存储用户地址列表 • 添加和删除用户 • 查询用户是否存在

✗ 不好的设计

```
address[] public users; // 可能无限增长

function removeUser(address user) public {
    for(uint i = 0; i < users.length; i++) {
        if(users[i] == user) {
            delete users[i]; // 留下空洞, 不改变长度
        }
    }
}
```

- users数组可能无限增长, 导致Gas成本不可控
- removeUser函数使用delete操作, 只会将元素重置为address(0), 而不会真正移除元素
- 查找用户需要遍历整个数组, 效率低下

✓ 好的设计

```
address[] public users; // 存储用户地址
mapping(address => bool) public isUser; // 快速查询用户是否存在
mapping(address => uint) public userIndex; // 用户索引映射
uint constant MAX_USERS = 1000;
```

```
function addUser(address user) public {
    require(!isUser[user], "User exists");
    require(users.length < MAX_USERS, "Array is full");
    users.push(user);
    userIndex[user] = users.length - 1;
    isUser[user] = true;
}
```

```
function removeUser(address user) public {
    require(isUser[user], "User not found");
    uint index = userIndex[user];
    address lastUser = users[users.length - 1];
    users[index] = lastUser;
    userIndex[lastUser] = index;
    users.pop();
    delete userIndex[user];
    isUser[user] = false;
}
```



知识点总结

📁 数组类型

- ✔ 定长数组: `uint[5]`, 长度固定
- ✔ 动态数组: `uint[]`, 长度可变

✂ 基本操作

- ➕ `push`: 添加元素
- ➖ `pop`: 删除最后元素
- 📏 `length`: 获取长度

🗑 删除元素

- ➡ 保持顺序: 移动元素
- ⚡ 快速删除: 替换+`pop`

🗪 多维数组

- `uint[][]`: 动态二维
- 🗪 `uint[3][4]`: 4个长度3的数组

⚡ Gas优化技巧

- 📦 缓存 `length`
- ⚡ 限制最大长度
- 📋 分批处理
- ↔ 使用 `calldata`
- 🚫 避免循环写storage



今日作业

1 安全数组管理合约

- ✓ 限制最大长度为 100
- ✓ 实现添加功能 (safePush)
- ✓ 实现删除功能 (两种方法)
- ✓ 实现分批求和 (sumRange)
- ✓ 实现查找和获取所有元素功能

2 优化函数

优化以下函数，至少节省 15% gas:

```
function process(uint[] memory values) public {  
    for(uint i = 0; i < values.length; i++) {  
        if(values[i] > 10) {  
            data.push(values[i]);  
        }  
    }  
}
```

💡 提示：使用 calldata、缓存 length、减少 storage 操作

3 思考题

- 1 为什么要限制数组最大长度?
- 2 delete arr[i] 和 arr.pop() 有什么区别?
- 3 uint[3][4] 表示什么?
- 4 什么时候用数组，什么时候用 mapping?

下节课预告

→ 第3.3课：映射和结构体

📖 mapping 的使用和限制

键值对存储 • mapping 的特性

📦 struct 的定义和操作

自定义类型 • struct 成员访问

🔗 mapping 和 struct 的组合

最强大的数据结构 • 实际应用场景

🔧 复杂数据结构设计

用户管理系统 • 最佳实践

✅ 核心要点回顾

- 动态数组可 push/pop
- delete 不改变 length
- 大数组遍历危险
- 必须缓存 length
- 限制最大长度至关重要
- 考虑用 mapping 替代

💡 理解数组特性 = 避免致命错误